



Design Document
AttireAI - Team 11

Team:

Yuanfei Song - song748@purdue.edu,
Yichen Dai - dai229@purdue.edu,
Congtian Wu - wu1908@purdue.edu,
Ekaterina Tsz Yao - etszyao@purdue.edu

Table of Contents:

Team 11 1

Purpose 3

- Functional Requirements 4
- Non-Functional Requirements 6

Design Outline 8

- High Level Overview 10
- Activity State Diagram 11

Design Issues 13

- Functional Issues 13
- Non-Functional Issues 15

Design Details 17

- Data Class Level - Design 17
- Database Design 18
- Description of database nodes 19

- Sequence of Events Diagram 21
- UI Mock-ups 24

Problem Statement

Online clothing shoppers frequently struggle with selecting the right size and style, leading to high return rates and customer dissatisfaction. While existing fashion apps offer basic size charts or generic style suggestions, they fail to provide a personalized, end-to-end solution that considers individual body measurements, skin tone, hair color, and occasion-specific needs. Our web application AttireAI can address this gap by combining AI-powered body measurement technology with intelligent style recommendations, allowing users to receive personalized outfit suggestions complete with pricing and direct purchase links. Unlike competitors such as Stitch Fix or Amazon StyleSnap, AttireAI offers live measurement capture, comprehensive color analysis, and a virtual try-on feature that generates realistic outfit previews on the user's own image.

E-commerce apparel shopping has a persistent “fit and style uncertainty” problem: users cannot reliably judge sizing and how an outfit will look on them using product photos alone. Body dimensions (e.g., chest, waist, hip, inseam), brand-to-brand sizing differences, and personal features such as skin tone and hair color strongly affect what fits well and what looks flattering. In addition, users shop for different social contexts (e.g., an interview, a formal dinner, a casual party), but most product pages do not translate those contexts into complete outfit choices.

Targeted users include:

- Online shoppers who frequently return clothing due to sizing issues or mismatched expectations.
- Fashion-conscious users who want guidance on what to wear for specific occasions.
- Users who want quick outfit bundles (top/bottom/shoes/accessories) instead of searching item-by-item.

Requirements

Functional Requirements

User Authentication & Account Management

ID	User Story	Hours
1	As a user, I would like to create an account using my email address so that I can save my preferences and measurements.	6
2	As a user, I would like to log in using social login options (Google, Microsoft, Apple) for faster access to my account.	6
3	As a user, I would like to reset my password via email if I forget it.	2
4	As a user, I would like to log out of my account securely from any device.	1
5	As a user, I would like to view and edit my profile information (name, email, profile picture).	4
6	As a user, I would like to delete my account and all associated data if I no longer wish to use the service.	2

Body Measurements

ID	User Story	Hours
7	As a user, I would like to manually input my body measurements (waist, chest, hip, inseam, shoulder width, arm length, height, weight) so the system can recommend properly fitting clothes.	6
8	As a user, I would like to use my device's camera to	36

	automatically capture my body measurements through live detection technology.	
9	As a user, I would like to receive on-screen guidance during the live measurement process to ensure accurate capture (e.g., "Stand 6 feet away", "Turn sideways").	8
10	As a user, I would like to review and confirm the automatically detected measurements before saving them.	2
11	As a user, I would like to edit individual measurements if the automatic detection was inaccurate.	3
12	As a user, I would like to save multiple measurement profiles (e.g., for different family members).	5
13	As a user, I would like the system to convert measurements between metric and imperial units based on my preference.	2
14	As a user, I would like to see which size I would be at different brands/retailers based on my measurements.	8

Color Analysis

ID	User Story	Hours
15	As a user, I would like to upload a photo of myself for skin tone analysis so that recommended outfits complement my complexion.	8
16	As a user, I would like to use my device's camera to capture my skin tone in real-time.	10
17	As a user, I would like the system to detect my hair color from an uploaded or captured photo.	6
18	As a user, I would like to manually select or adjust my skin tone category if the automatic detection is inaccurate.	2

19	As a user, I would like to manually select or adjust my hair color if the automatic detection is inaccurate.	2
20	As a user, I would like to see my color profile summary (skin tone, hair color, recommended color palette) in my account.	4
21	As a user, I would like to receive recommendations for clothing colors that complement my skin tone and hair color.	6

Style Preferences

ID	User Story	Hours
22	As a user, I would like to select my preferred fashion styles from a list (minimalist, streetwear, classic, bohemian, preppy, athleisure, vintage, etc.) so recommendations match my taste.	4
23	As a user, I would like to select multiple style preferences to receive diverse outfit suggestions.	2
24	As a user, I would like to indicate styles I want to avoid so they are excluded from recommendations.	2
25	As a user, I would like to take a visual style quiz where I choose between outfit images to help the system understand my preferences.	12
26	As a user, I would like to update my style preferences at any time.	1
27	As a user, I would like to set preferred price ranges (budget, mid-range, luxury) for outfit recommendations.	3
28	As a user, I would like to specify preferred brands or retailers I like shopping from.	3
29	As a user, I would like to specify brands or retailers I want to exclude from recommendations.	2

Occasion Selection

ID	User Story	Hours
30	As a user, I would like to select an occasion type (formal dinner, casual party, business meeting, date night, wedding guest, job interview, workout, everyday casual, etc.) when requesting outfit recommendations.	4
31	As a user, I would like to specify the weather/season for the occasion so recommendations are weather-appropriate.	4
32	As a user, I would like to specify a dress code if applicable (black tie, business casual, smart casual, etc.).	2
33	As a user, I would like to save frequently used occasions as presets for quick access.	4
34	As a user, I would like to add custom occasion types if my specific need isn't listed.	2

AI Outfit Recommendations

ID	User Story	Hours
35	As a user, I would like to receive complete outfit recommendations based on my measurements, color profile, style preferences, and selected occasion.	25
36	As a user, I would like each outfit recommendation to include multiple clothing items that work together (top, bottom, shoes, accessories).	8
37	As a user, I would like to see multiple outfit options (at least 5) for each recommendation request so I can choose my favorite.	6
38	As a user, I would like to view the reasoning behind each recommendation (e.g., "This color complements your skin tone", "This cut flatters your body type").	8
39	As a user, I would like to rate outfit	6

	recommendations (like/dislike) so the system can learn my preferences over time.	
40	As a user, I would like to request new recommendations if I don't like the current suggestions.	2
41	As a user, I would like to filter recommendations by specific criteria (color, price, brand, item type).	6
42	As a user, I would like to see alternative items for any piece in a recommended outfit (e.g., "Show me other pants that would work with this outfit").	8
43	As a user, I would like the system to learn from my rating history and improve recommendations over time.	12

Product Information & Purchase

ID	User Story	Hours
44	As a user, I would like to see detailed product information for each recommended item (name, brand, material, care instructions).	4
45	As a user, I would like to see the price for each recommended item.	2
46	As a user, I would like to see which size I should order for each item based on my measurements.	6
47	As a user, I would like to click a direct purchase link that takes me to the retailer's website to buy the item.	4
48	As a user, I would like to see the total estimated cost of a complete outfit.	2
49	As a user, I would like to see if recommended items are in stock in my size.	6
50	As a user, I would like to be notified when items in my saved outfits go on sale.	8
51	As a user, I would like to compare prices across	8

	different retailers for the same or similar items.	
--	--	--

Saved Items & History

ID	User Story	Hours
52	As a user, I would like to save outfit recommendations to a favorites/wishlist for future reference.	4
53	As a user, I would like to organize saved outfits into custom collections (e.g., "Work Outfits", "Weekend Looks", "Special Events").	6
54	As a user, I would like to view my recommendation history to revisit past suggestions.	4
55	As a user, I would like to remove items from my saved collections.	1
56	As a user, I would like to share saved outfits with friends via social media or messaging.	6
57	As a user, I would like to mark items I've already purchased to track my wardrobe.	4
58	As a user, I would like to receive outfit suggestions using items I've already purchased/marked as owned.	12

Virtual Try-On - VIP Feature (VIRTUAL TRY-ON)

ID	User Story	Hours
59	As a VIP user, I would like to upload a full-body photo of myself for virtual try-on purposes.	6
60	As a VIP user, I would like to see realistic preview images of recommended outfits rendered on my uploaded photo.	35
61	As a VIP user, I would like to try on individual items on my photo, not just complete outfits.	12
62	As a VIP user, I would like to save virtual try-on	4

	images to my account for later review.	
63	As a VIP user, I would like to compare multiple virtual try-on images side by side.	6
64	As a VIP user, I would like to share my virtual try-on images with friends for feedback.	4
65	As a VIP user, I would like to view virtual try-on results from different angles if multiple photos are provided.	15

VIP Subscription

ID	User Story	Hours
66	As a user, I would like to view the benefits of VIP subscription before subscribing.	2
67	As a user, I would like to subscribe to VIP membership to access virtual try-on and premium features.	8
68	As a VIP user, I would like to manage my subscription (view status, cancel, update payment method).	6
69	As a VIP user, I would like to receive unlimited outfit recommendations (vs. limited for free users).	4
70	As a user, I would like to receive a free trial of VIP features before committing to a subscription.	6

User Interface & Navigation (UI/UX)

ID	User Story	Hours
71	As a user, I would like a clean, intuitive homepage that guides me through the recommendation process.	8
72	As a user, I would like to access all main features from a clear navigation menu.	4

73	As a user, I would like the website to be responsive and work well on both desktop and mobile browsers.	12
74	As a user, I would like to see loading indicators when the system is processing my requests.	2
75	As a user, I would like to receive clear error messages if something goes wrong.	3
76	As a user, I would like to access a help/FAQ section to learn how to use the application.	4
77	As a user, I would like to complete an onboarding tutorial when I first create an account.	6
78	As a user, I would like to switch between light and dark mode themes.	4

Admin Features

ID	User Story	Hours
79	As an administrator, I would like to view user statistics and analytics (total users, active users, popular features).	8
80	As an administrator, I would like to manage the product database (add, edit, remove clothing items).	10
81	As an administrator, I would like to manage retailer partnerships and affiliate links.	6
82	As an administrator, I would like to review and respond to user feedback/support requests.	4

Non-Functional Requirements

Usability

- The application will have an intuitive, user-friendly interface with clear navigation across all main features (measurements, color analysis, outfit recommendations, virtual try-on).

- The application will be fully responsive and optimized for various device sizes (desktop, tablet, mobile) to ensure a consistent experience across platforms.
- The application will display helpful error messages and real-time guidance during critical workflows, particularly during live measurement capture and photo uploads.
- The application will provide clear visual feedback through loading indicators and progress bars during AI processing tasks.

Performance

- The homepage and primary navigation pages will load in under 3 seconds to maintain user engagement.
- AI outfit recommendations will be generated and displayed within 10 seconds of user request submission.
- Body measurement detection via camera will process and return results within 15 seconds to ensure a smooth user experience.
- Virtual try-on image generation will complete within 20 seconds to balance quality with acceptable wait times.
- The application will implement efficient caching strategies to reduce load times for frequently accessed data, such as saved outfits and user profiles.
- Image uploads will be optimized and compressed to ensure fast processing without sacrificing quality.

Scalability

- The application will be deployed on AWS cloud infrastructure that supports automatic scaling based on user demand and traffic patterns.
- The database architecture will be designed to handle growth from hundreds of concurrent users without performance degradation.
- The FastAPI/Flask backend will support horizontal scaling, allowing multiple instances to run simultaneously to handle increased load.
- The system will implement efficient data storage and retrieval practices to manage growing volumes of user profiles, measurement data, and outfit history.
- The application architecture will be modular to allow independent scaling of resource-intensive components like AI processing and image generation.

Security

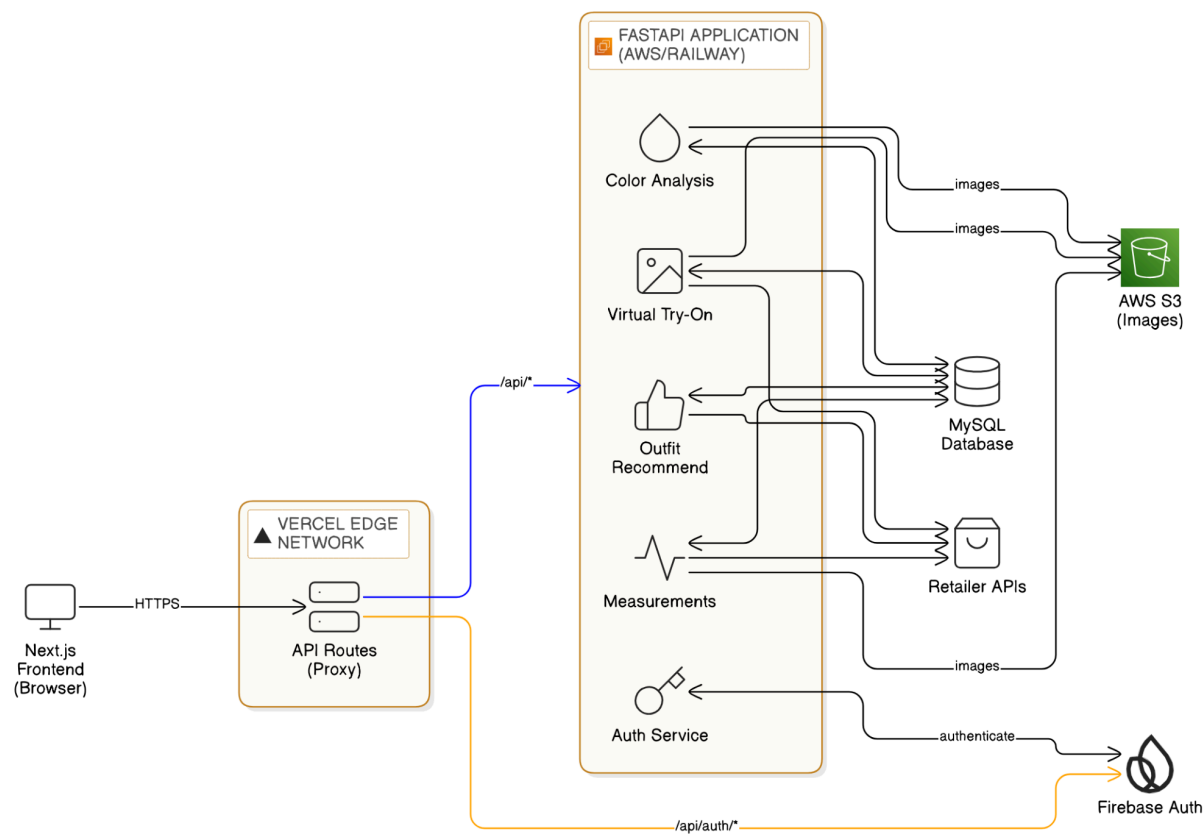
- All communication between client and server will be encrypted using HTTPS/TLS to protect data in transit.

- User passwords will be hashed using bcrypt or similar industry-standard algorithms before storage, never stored in plain text.
- The application will implement secure session management with JWT tokens that expire after a period of inactivity.
- Access control will be enforced at the API level, ensuring users can only access and modify their own data through proper authentication and authorization checks.
- Uploaded photos and sensitive user data will be stored in secure AWS S3 buckets with appropriate access policies and encryption at rest.
- The application will implement input validation and sanitization on all user inputs to prevent SQL injection, XSS, and other common vulnerabilities.
- Payment processing for VIP subscriptions will be handled through PCI-compliant third-party payment processors (e.g., Stripe) to ensure no sensitive payment data is stored on our servers.
- User account deletion will permanently remove all associated personal data, including measurements, photos, and preferences, from the system.
- The FastAPI/Flask backend will implement rate limiting to prevent brute force attacks and API abuse.

2. Design Outline

2.1 Architecture Overview

AttireAI uses a **client-server architecture** with a Next.js frontend hosted on Vercel, a FastAPI Python backend for AI processing, and MySQL for persistent storage.

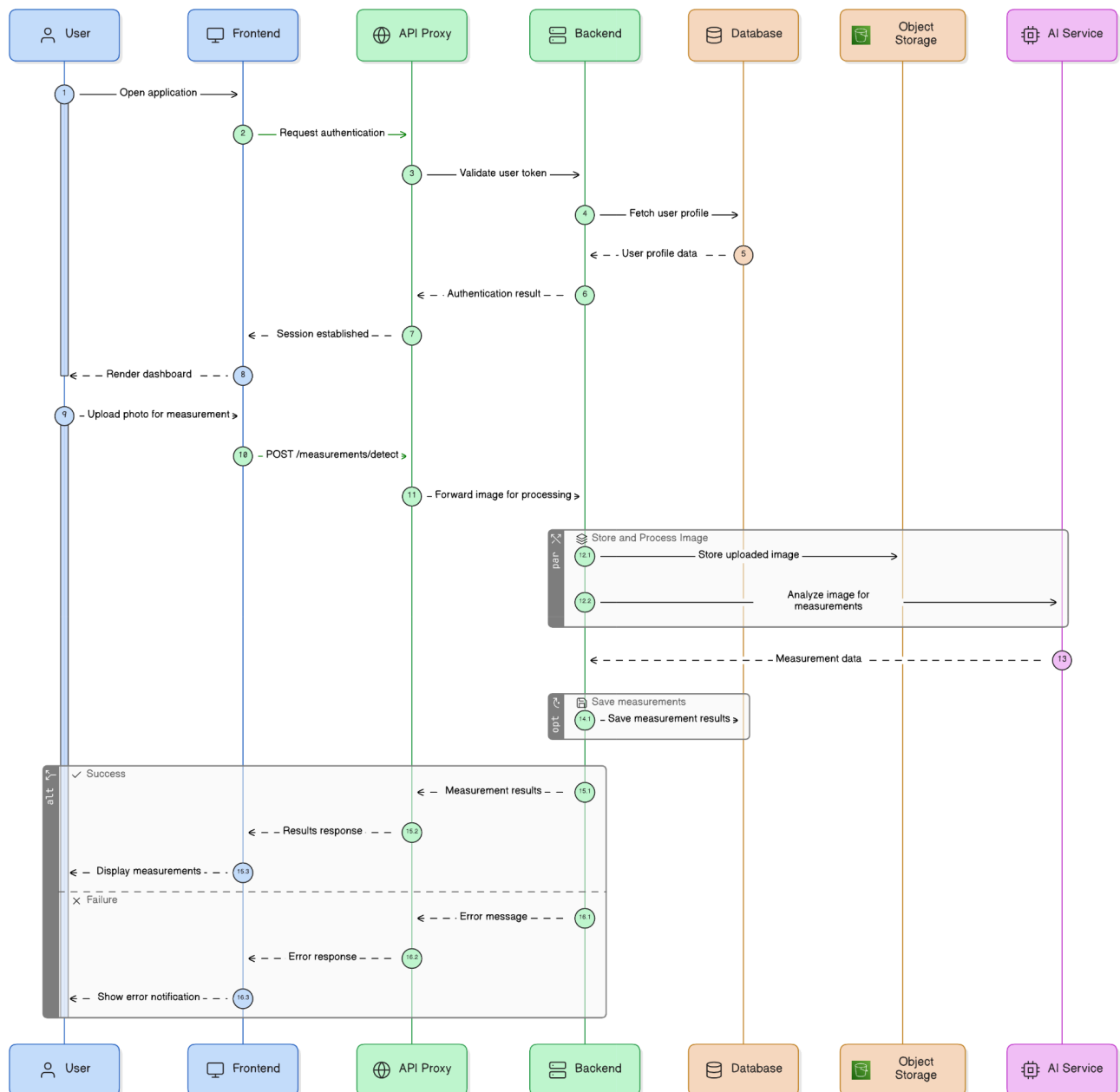


2.2 System Components

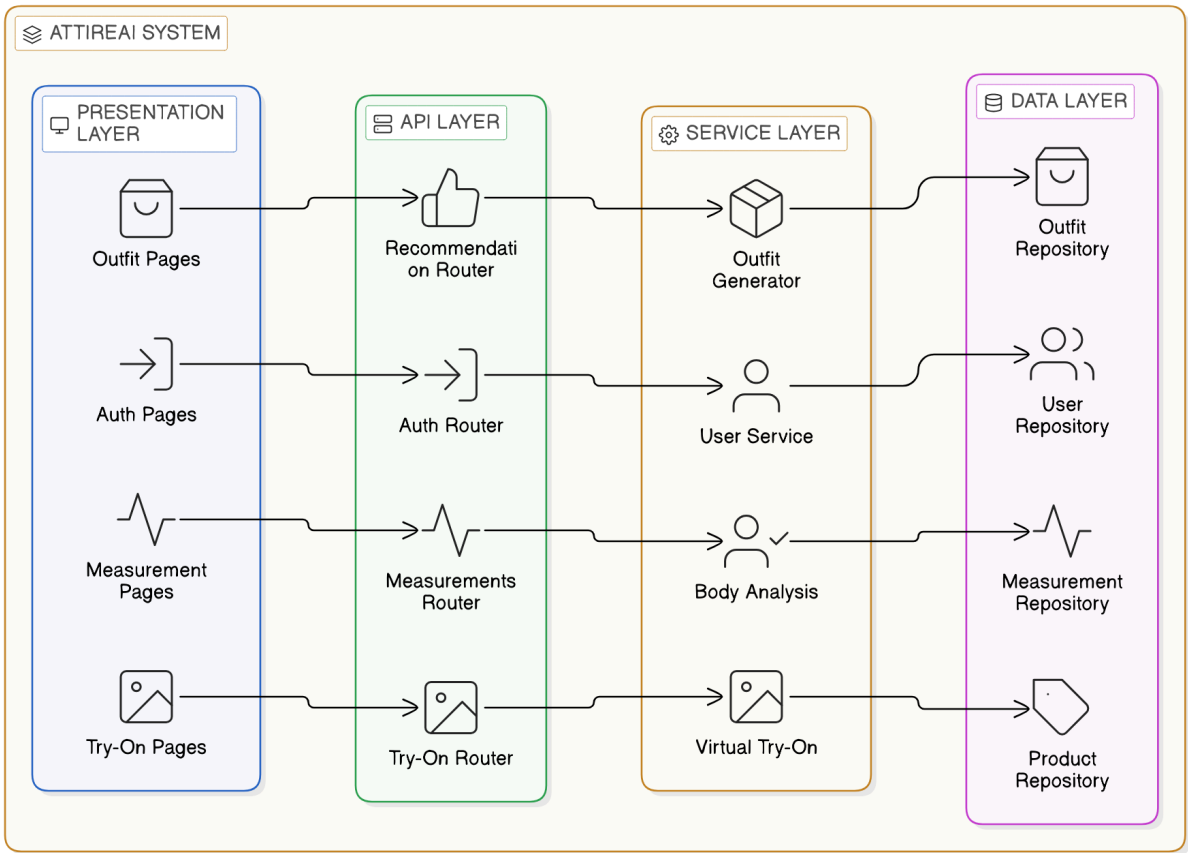
Component	Technology	Purpose
-----	-----	-----
Frontend	Next.js 14, TypeScript	Responsive UI, SSR, client-side routing
API Proxy	Next.js API Routes	Route requests to backend, handle auth cookies
Backend	FastAPI	REST API, AI/ML processing, business logic
Database	MySQL 8.0	User data, measurements, preferences, history

Auth	Firebase Auth	Email/password, social login providers
Storage	AWS S3	User photos, virtual try-on images
ML Models	PyTorch, OpenCV	Body measurement, color analysis, virtual try-on
Hosting	Vercel, AWS/Railway	Scalable deployment

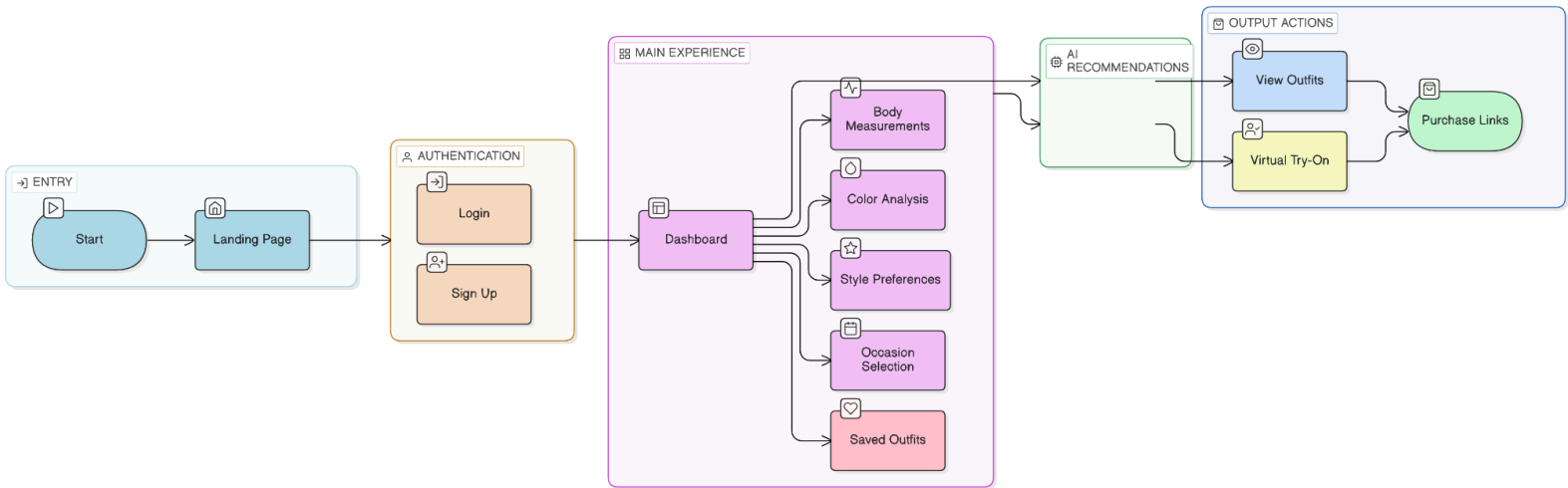
2.3 Component Interactions



2.4 High-Level UML Component Diagram



2.5 Activity State Diagram - User Flow



3. Design Issues

3.1 Functional Issues

Issue 1: How should users provide body measurements?

Option	Description	Pros	Cons
--------	-------------	------	------

-----	-----	-----	-----
-------	-------	-------	-------

Option 1	Manual input only	Simple, reliable, no camera needed	Tedious, users may not know measurements
----------	-------------------	------------------------------------	--

Option 2	Camera detection only	Convenient, automatic	Requires good lighting, privacy concerns
----------	-----------------------	-----------------------	--

Option 3	Both manual and camera	Flexibility, users can verify/correct	More development effort
-----------------	-------------------------------	--	--------------------------------

Decision: Option 3 - Both manual input and camera detection**

Justification: Providing both options maximizes accessibility and accuracy. Users who know their measurements can input them directly, while others can use the camera feature. The ability to review and edit detected measurements ensures accuracy while maintaining convenience. This approach addresses our core value proposition of reducing sizing uncertainty.

Issue 2: How should the AI recommendation system work?

Option	Description	Pros	Cons
--------	-------------	------	------

-----	-----	-----	-----
-------	-------	-------	-------

Option 1	Rule-based system	Predictable, explainable	Limited flexibility, hard to scale
----------	-------------------	--------------------------	------------------------------------

| Option 2 | Pure ML model | Highly personalized | Black box, needs lots of data |
| **Option 3** | **Hybrid (rules + ML)** | **Explainable + adaptive** | **More complex architecture** |

Decision: Option 3 - Hybrid approach (rules + ML)

Justification: A hybrid system uses rule-based logic for fundamental fashion principles (color theory, occasion appropriateness, body type guidelines) while employing ML for personalization based on user feedback. This ensures recommendations are always reasonable while improving over time. The rule-based component also allows us to explain recommendations to users ("This color complements your skin tone").

Issue 3: How should virtual try-on be implemented?

Option	Description	Pros	Cons
Option 1	2D overlay compositing	Fast, simple	Less realistic
Option 2	3D body model rendering	Highly realistic	Computationally expensive, complex
Option 3	**AI image generation (diffusion)**	**Realistic, flexible**	**GPU-intensive, potential artifacts**

Decision: Option 3 - AI image generation using diffusion models**

Justification: Modern diffusion models (like Stable Diffusion with ControlNet) can generate highly realistic try-on images while handling various poses and lighting conditions. Though computationally intensive, we can optimize by running inference on GPU-enabled cloud instances and caching results. This is reserved for VIP users, making the cost manageable.

Issue 4: How should user authentication be handled?

Option	Description	Pros	Cons

| Option 1 | Custom auth system | Full control | Security risks, more dev time |
| **Option 2 | Firebase Authentication | Secure, social login support | Vendor dependency |**
| Option 3 | Auth0 / Clerk | Feature-rich | Cost at scale |

Decision: Option 2 - Firebase Authentication**

Justification: Firebase Auth provides secure, battle-tested authentication with built-in support for email/password and social logins (Google, Microsoft, Apple). It integrates well with our tech stack, has a generous free tier, and the team has prior experience with Firebase from CS 307. This reduces development time while ensuring security.

Issue 5: How should product data be sourced?

Option	Description	Pros	Cons
Option 1	Build own product database	Full control	Massive data entry effort
**Option 2	Retailer API integrations	Real-time data, direct links	API limitations, partnerships needed
Option 3	Affiliate network APIs	Easy integration, wide selection	Less control over data quality

Decision: Option 2 with Option 3 fallback - Retailer APIs with affiliate network backup**

Justification: We will prioritize direct retailer API integrations where available for accurate, real-time product data. For broader coverage, we'll supplement with affiliate network APIs (e.g., ShopStyle, Rakuten). This hybrid approach ensures we have sufficient product variety while maintaining data quality for key partnerships.

Issue 6: How should outfit recommendations be presented?

| Option | Description | Pros | Cons |

|-----|-----|-----|-----|

| Option 1 | Single best outfit | Simple, decisive | Users want choices |

| **Option 2** | **Multiple complete outfits** | **Variety, user choice** | **May overwhelm** |

| Option 3 | Mix-and-match components | Maximum flexibility | Complexity, decision fatigue |

Decision: Option 2 - Multiple complete outfits (5+ options)**

Justification: Presenting 5+ complete outfit recommendations gives users meaningful choice without overwhelming them. Each outfit is a cohesive bundle (top, bottom, shoes, accessories), reducing decision fatigue compared to mix-and-match. Users can also request alternatives for individual pieces within an outfit.

3.2 Non-Functional Issues

Issue 1: What frontend framework should be used?

| Option | Description | Pros | Cons |

|-----|-----|-----|-----|

| Option 1 | React (CRA/Vite) | Simple, flexible | No SSR out of box |

| **Option 2** | **Next.js** | **SSR, file-based routing, Vercel integration** | **Learning curve** |

| Option 3 | Vue.js / Nuxt | Simple syntax | Smaller ecosystem for our needs |

Decision: Option 2 - [Next.js](#)

Justification: Next.js provides server-side rendering for better SEO and initial load performance, API routes for proxying backend requests, and seamless Vercel deployment. The App Router (Next.js 14) offers excellent developer experience with React Server Components. TypeScript support ensures type safety across the codebase.

Issue 2: What Python backend framework should be used?

Option	Description	Pros	Cons
Option 1	Flask	Simple, lightweight, flexible	Sync by default, manual docs
Option 2	FastAPI	Async, auto OpenAPI docs, Pydantic	Newer ecosystem
Option 3	Django	Batteries-included, ORM	Overkill for API-only

Decision: Option 2 - FastAPI

Justification: FastAPI's async support is crucial for handling concurrent image processing requests (body detection, virtual try-on). Automatic OpenAPI documentation aids frontend development. Pydantic models ensure request/response validation. High performance is comparable to Node.js/Go, and it integrates seamlessly with ML libraries (PyTorch, OpenCV).

Issue 3: What database system should be used?

Option	Description	Pros	Cons
Option 1	PostgreSQL	Robust, JSON support, scalable	More complex setup
Option 2	MySQL	Familiar, reliable, good tooling	Less flexible than Postgres
Option 3	MongoDB	Flexible schema, good for varied data	Consistency concerns

Decision: Option 2 - MySQL

Justification: MySQL is specified in our project requirements and provides reliable relational data storage for user profiles, measurements, and preferences. Our data model is well-defined and relational (users have measurements, preferences, saved outfits), making SQL appropriate. MySQL's wide support ensures easy deployment on various hosting platforms.

Issue 4: How should the backend be hosted?

Option	Description	Pros	Cons
--------	-------------	------	------

-----	-----	-----	-----
-------	-------	-------	-------

Option 1	AWS EC2	Full control, GPU instances	Complex management
-----------------	----------------	------------------------------------	---------------------------

Option 2	Railway / Render	Simple deployment, auto-scaling	Less control, potential cost
----------	------------------	---------------------------------	------------------------------

Option 3	Vercel Serverless	Same platform as frontend	Python cold starts, timeout limits
----------	-------------------	---------------------------	------------------------------------

Decision: Option 1 (AWS) for AI workloads, Option 2 (Railway) for API

Justification: We'll use a split approach: Railway for the main FastAPI application (simple deployment, auto-scaling) and AWS EC2/Lambda with GPU for intensive AI tasks (body detection, virtual try-on). This balances ease of deployment with the computational requirements of ML inference.

Issue 5: How should images be stored?

Option	Description	Pros	Cons
--------	-------------	------	------

-----	-----	-----	-----
-------	-------	-------	-------

Option 1	Database BLOBs	Simple, single source	Performance issues, DB bloat
----------	----------------	-----------------------	------------------------------

Option 2	AWS S3	Scalable, CDN-ready, secure	Additional service
-----------------	---------------	------------------------------------	---------------------------

Option 3	Cloudinary	Image optimization built-in	Cost at scale
----------	------------	-----------------------------	---------------

Decision: Option 2 - AWS S3

Justification: S3 provides secure, scalable image storage with fine-grained access control. User photos (for measurements and try-on) are sensitive data, and S3's encryption at rest and IAM policies ensure security. CloudFront CDN integration enables fast image delivery globally.

Issue 6: How should the frontend communicate with the backend?

Option	Description	Pros	Cons
--------	-------------	------	------

|-----|-----|-----|-----|

| Option 1 | Direct API calls | Simpler, faster | CORS issues, exposed backend URL |

| **Option 2** | **Next.js API routes as proxy** | **Hidden backend, cookie handling** | **Extra hop** |

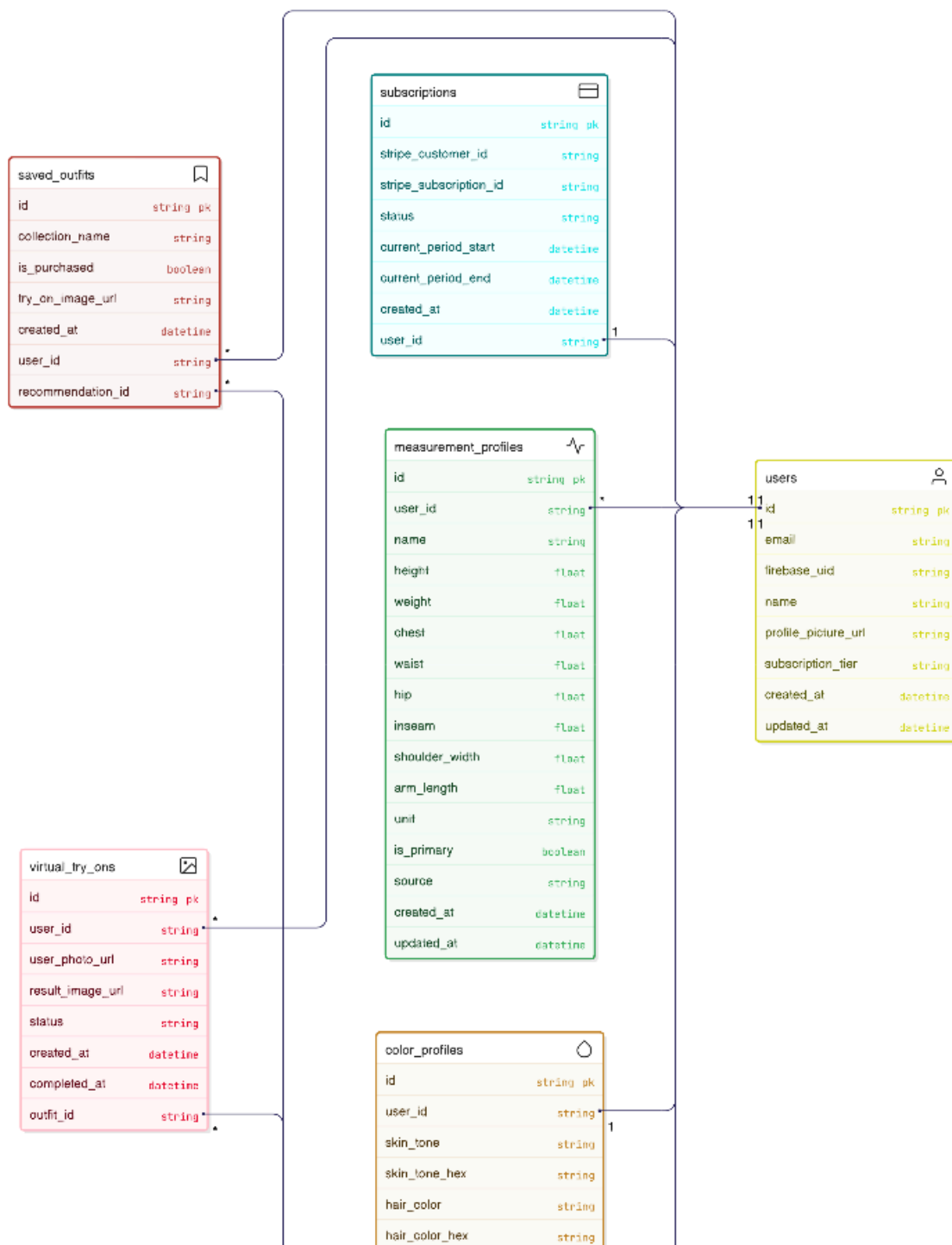
| Option 3 | GraphQL | Flexible queries | Overhead for our use case |

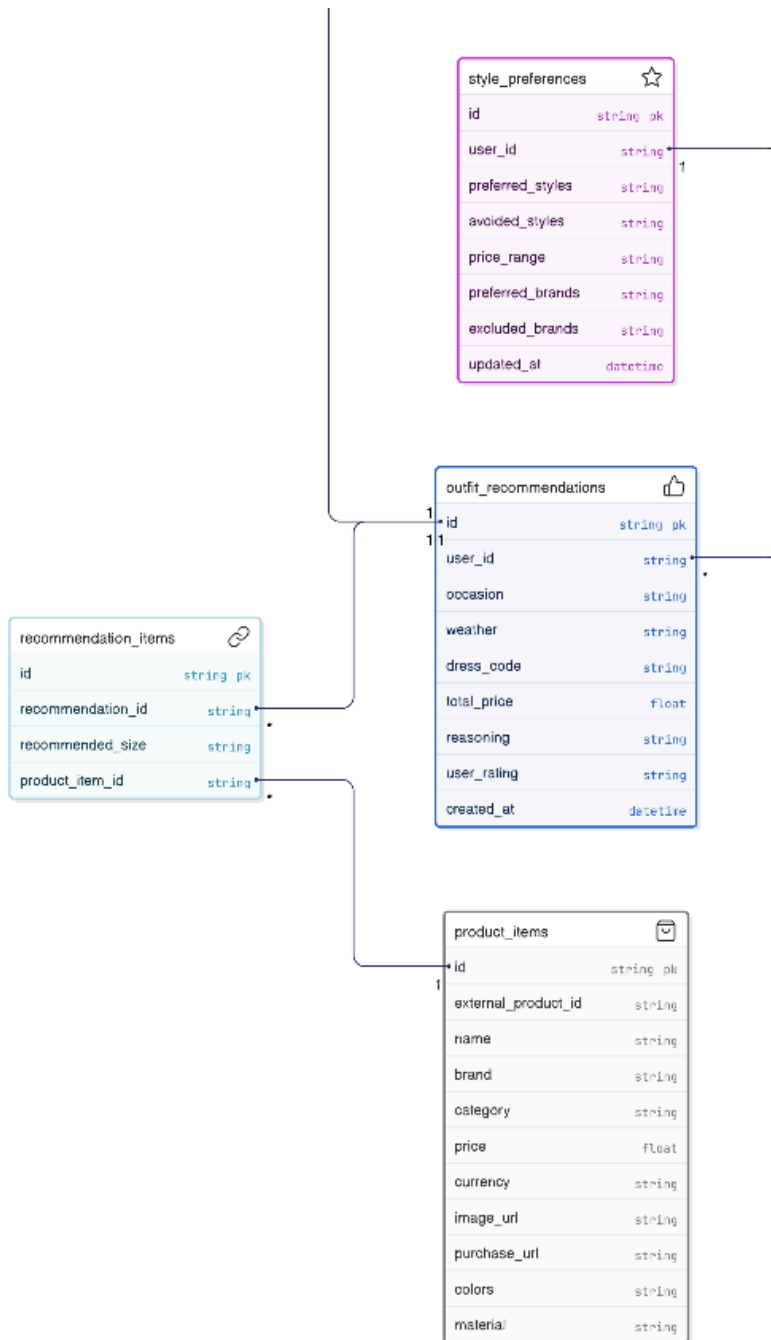
Decision: Option 2 - Next.js API routes as proxy

Justification: Using Next.js API routes as a proxy layer keeps our FastAPI backend URL private, handles authentication cookies seamlessly, and avoids CORS configuration. The minimal latency overhead is acceptable given the benefits of centralized request handling and the ability to add rate limiting or caching at the proxy layer.

4. Design Details

4.1 Class Diagram - Core Domain Models





4.2 Database Schema (MySQL)

-- Users table

```
CREATE TABLE users (  
  id CHAR(36) PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  firebase_uid VARCHAR(128) UNIQUE NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  profile_picture_url VARCHAR(500),  
  subscription_tier ENUM('FREE', 'VIP') DEFAULT 'FREE',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  INDEX idx_firebase_uid (firebase_uid),  
  INDEX idx_email (email)  
);
```

-- Measurement profiles (users can have multiple)

```
CREATE TABLE measurement_profiles (  
  id CHAR(36) PRIMARY KEY,  
  user_id CHAR(36) NOT NULL,  
  name VARCHAR(50) NOT NULL,  
  height DECIMAL(5,2),  
  weight DECIMAL(5,2),  
  chest DECIMAL(5,2),  
  waist DECIMAL(5,2),  
  hip DECIMAL(5,2),  
  inseam DECIMAL(5,2),  
  shoulder_width DECIMAL(5,2),  
  arm_length DECIMAL(5,2),  
  unit ENUM('CM', 'IN') DEFAULT 'CM',  
  is_primary BOOLEAN DEFAULT FALSE,  
  source ENUM('MANUAL', 'CAMERA') DEFAULT 'MANUAL',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
INDEX idx_user_id (user_id)  
);
```

-- Color profiles

```
CREATE TABLE color_profiles (  
  id CHAR(36) PRIMARY KEY,  
  user_id CHAR(36) UNIQUE NOT NULL,  
  skin_tone VARCHAR(50),  
  skin_tone_hex VARCHAR(7),  
  hair_color VARCHAR(50),  
  hair_color_hex VARCHAR(7),  
  recommended_palette JSON,  
  photo_url VARCHAR(500),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

-- Style preferences

```
CREATE TABLE style_preferences (  
  id CHAR(36) PRIMARY KEY,  
  user_id CHAR(36) UNIQUE NOT NULL,  
  preferred_styles JSON,  
  avoided_styles JSON,  
  price_range ENUM('BUDGET', 'MID_RANGE', 'LUXURY') DEFAULT 'MID_RANGE',  
  preferred_brands JSON,  
  excluded_brands JSON,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

-- Outfit recommendations

```
CREATE TABLE outfit_recommendations (  
  id CHAR(36) PRIMARY KEY,  
  user_id CHAR(36) NOT NULL,  
  occasion VARCHAR(100),  
  weather VARCHAR(50),  
  dress_code VARCHAR(50),  
  total_price DECIMAL(10,2),  
  reasoning TEXT,  
  user_rating ENUM('LIKE', 'DISLIKE', 'NONE') DEFAULT 'NONE',
```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_created (user_id, created_at)
);

-- Product items in recommendations
CREATE TABLE recommendation_items (
    id CHAR(36) PRIMARY KEY,
    recommendation_id CHAR(36) NOT NULL,
    external_product_id VARCHAR(100),
    name VARCHAR(255) NOT NULL,
    brand VARCHAR(100),
    category ENUM('TOP', 'BOTTOM', 'SHOES', 'ACCESSORY', 'OUTERWEAR'),
    price DECIMAL(10,2),
    currency VARCHAR(3) DEFAULT 'USD',
    image_url VARCHAR(500),
    purchase_url VARCHAR(500),
    recommended_size VARCHAR(20),
    colors JSON,
    material VARCHAR(100),
    FOREIGN KEY (recommendation_id) REFERENCES outfit_recommendations(id) ON DELETE
    CASCADE,
    INDEX idx_recommendation_id (recommendation_id)
);

-- Saved outfits / collections
CREATE TABLE saved_outfits (
    id CHAR(36) PRIMARY KEY,
    user_id CHAR(36) NOT NULL,
    recommendation_id CHAR(36) NOT NULL,
    collection_name VARCHAR(50) DEFAULT 'Favorites',
    is_purchased BOOLEAN DEFAULT FALSE,
    try_on_image_url VARCHAR(500),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (recommendation_id) REFERENCES outfit_recommendations(id) ON DELETE
    CASCADE,
    INDEX idx_user_collection (user_id, collection_name)
);

-- Virtual try-on requests (VIP only)
CREATE TABLE virtual_try_ons (
    id CHAR(36) PRIMARY KEY,
    user_id CHAR(36) NOT NULL,

```

```
user_photo_url VARCHAR(500) NOT NULL,  
outfit_id CHAR(36) NOT NULL,  
result_image_url VARCHAR(500),  
status ENUM('PENDING', 'PROCESSING', 'COMPLETED', 'FAILED') DEFAULT 'PENDING',  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
completed_at TIMESTAMP,  
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
FOREIGN KEY (outfit_id) REFERENCES outfit_recommendations(id) ON DELETE CASCADE,  
INDEX idx_user_status (user_id, status)  
);
```

-- VIP subscriptions

```
CREATE TABLE subscriptions (  
  id CHAR(36) PRIMARY KEY,  
  user_id CHAR(36) UNIQUE NOT NULL,  
  stripe_customer_id VARCHAR(100),  
  stripe_subscription_id VARCHAR(100),  
  status ENUM('ACTIVE', 'CANCELLED', 'PAST_DUE', 'TRIALING') DEFAULT 'TRIALING',  
  current_period_start TIMESTAMP,  
  current_period_end TIMESTAMP,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

4.3 API Endpoints (FastAPI)

```
``python
# Authentication
POST /api/auth/register      # Register new user
POST /api/auth/login         # Login with Firebase token
POST /api/auth/logout        # Logout user
DELETE /api/auth/account      # Delete user account

# User Profile
GET /api/users/me            # Get current user profile
PUT /api/users/me            # Update user profile
GET /api/users/me/dashboard   # Get dashboard summary

# Measurements
GET /api/measurements        # List measurement profiles
POST /api/measurements        # Create measurement profile (manual)
POST /api/measurements/detect # Detect measurements from image
GET /api/measurements/{id}    # Get specific profile
PUT /api/measurements/{id}    # Update measurement profile
DELETE /api/measurements/{id} # Delete measurement profile

# Color Analysis
GET /api/color-profile        # Get user's color profile
POST /api/color-profile/analyze # Analyze color from image
PUT /api/color-profile        # Update color profile manually

# Style Preferences
GET /api/preferences          # Get style preferences
PUT /api/preferences          # Update style preferences
GET /api/preferences/quiz     # Get style quiz questions
POST /api/preferences/quiz    # Submit style quiz answers

# Outfit Recommendations
POST /api/recommendations     # Generate new recommendations
GET /api/recommendations      # List past recommendations
GET /api/recommendations/{id} # Get specific recommendation
```

POST /api/recommendations/{id}/rate # Rate a recommendation
POST /api/recommendations/{id}/alternatives # Get alternative items

Saved Outfits

GET /api/saved-outfits # List saved outfits
POST /api/saved-outfits # Save an outfit
GET /api/saved-outfits/collections # List collections
PUT /api/saved-outfits/{id} # Update saved outfit
DELETE /api/saved-outfits/{id} # Remove saved outfit

Virtual Try-On (VIP)

POST /api/try-on # Create try-on request
GET /api/try-on/{id} # Get try-on status/result
GET /api/try-on # List user's try-on history

Subscription (VIP)

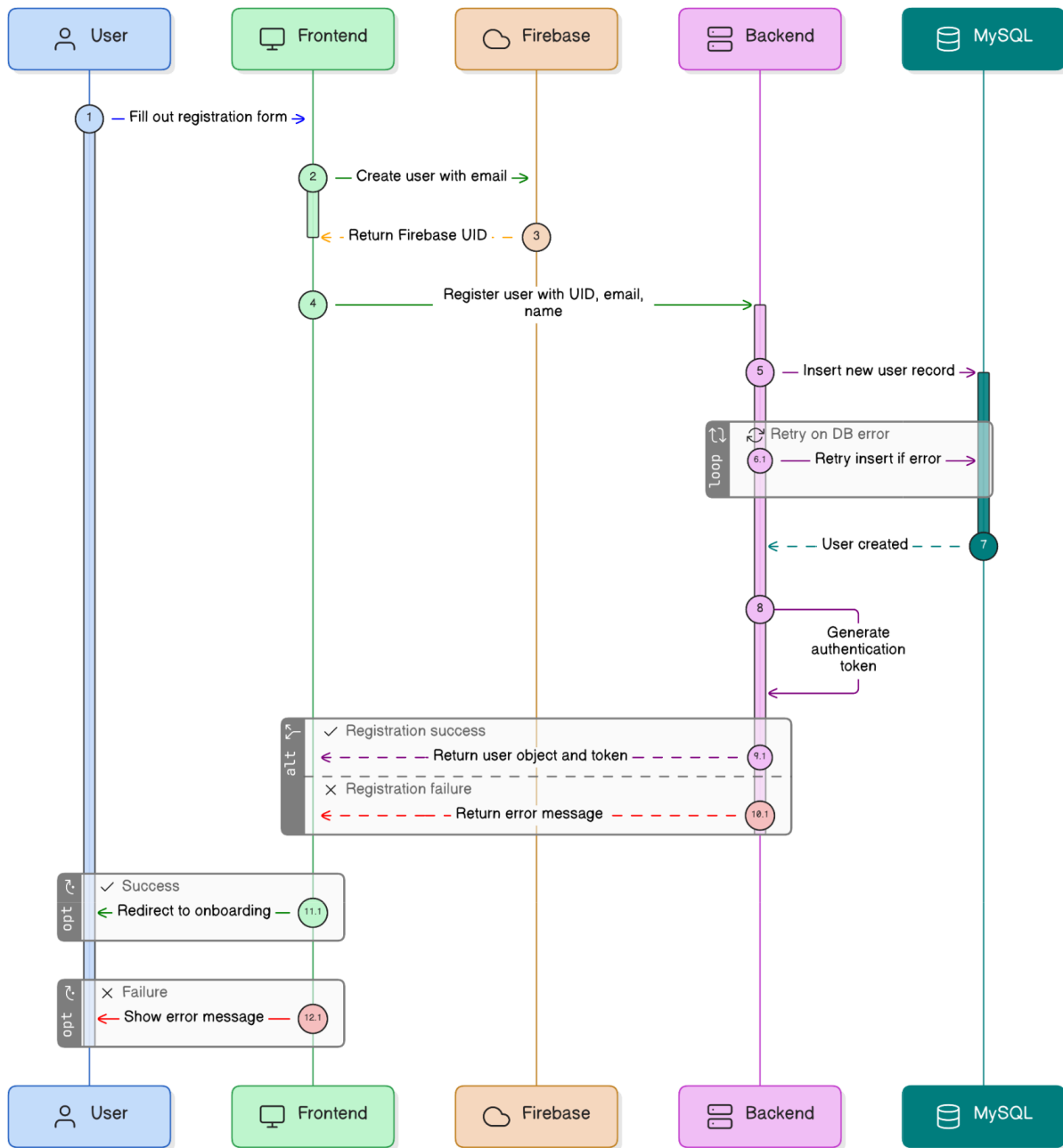
GET /api/subscription # Get subscription status
POST /api/subscription # Create subscription
DELETE /api/subscription # Cancel subscription
POST /api/subscription/portal # Get Stripe customer portal URL

Products (Internal)

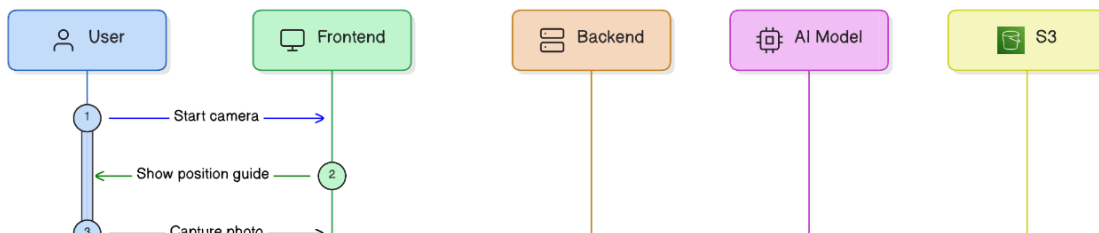
GET /api/products/search # Search products
GET /api/products/{id} # Get product details
GET /api/products/{id}/stock # Check stock status
``

4.4 Sequence Diagrams

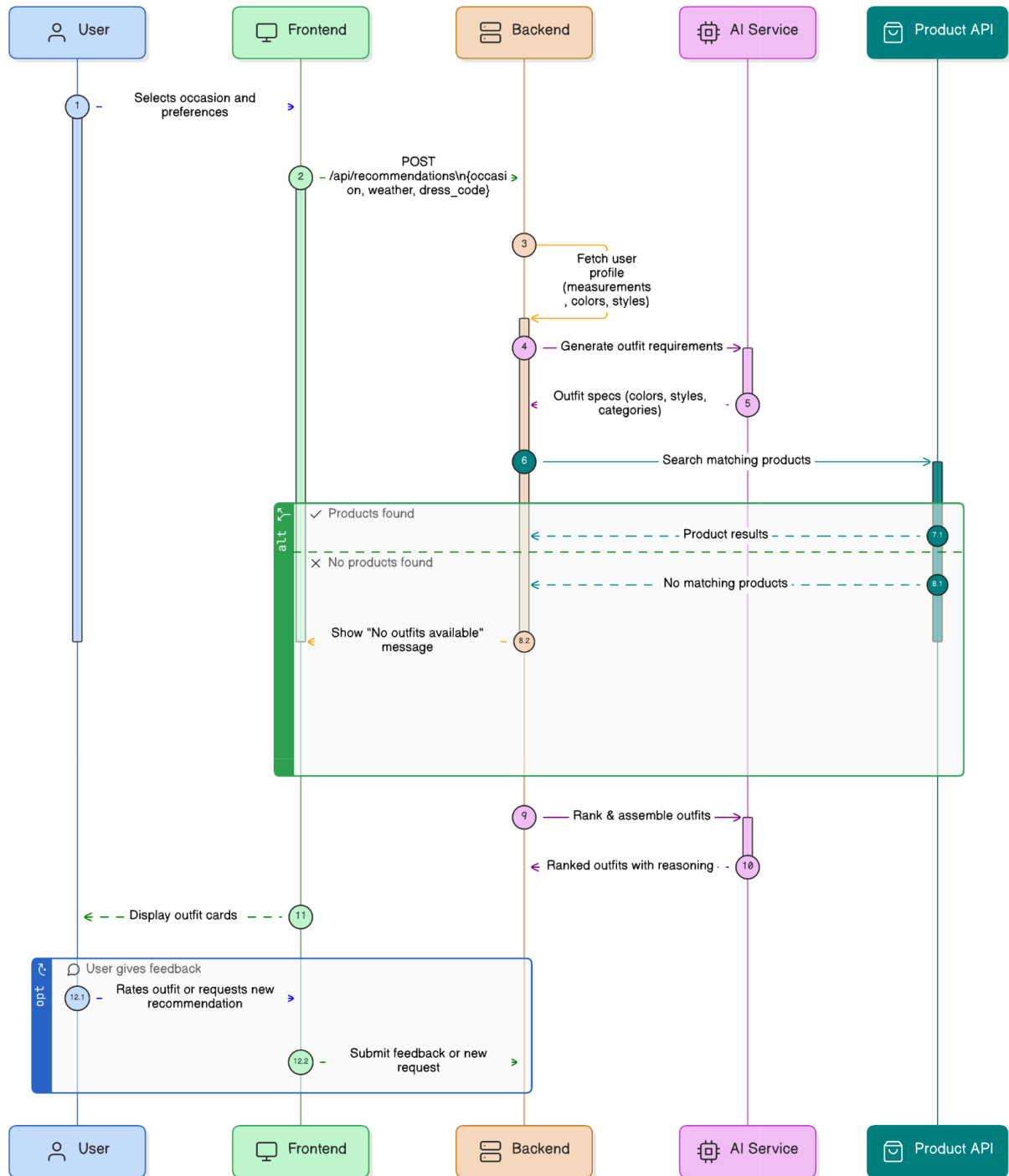
4.4.1 User Registration Flow



4.4.2 Body Measurement Detection Flow



4.4.3 Outfit Recommendation Flow

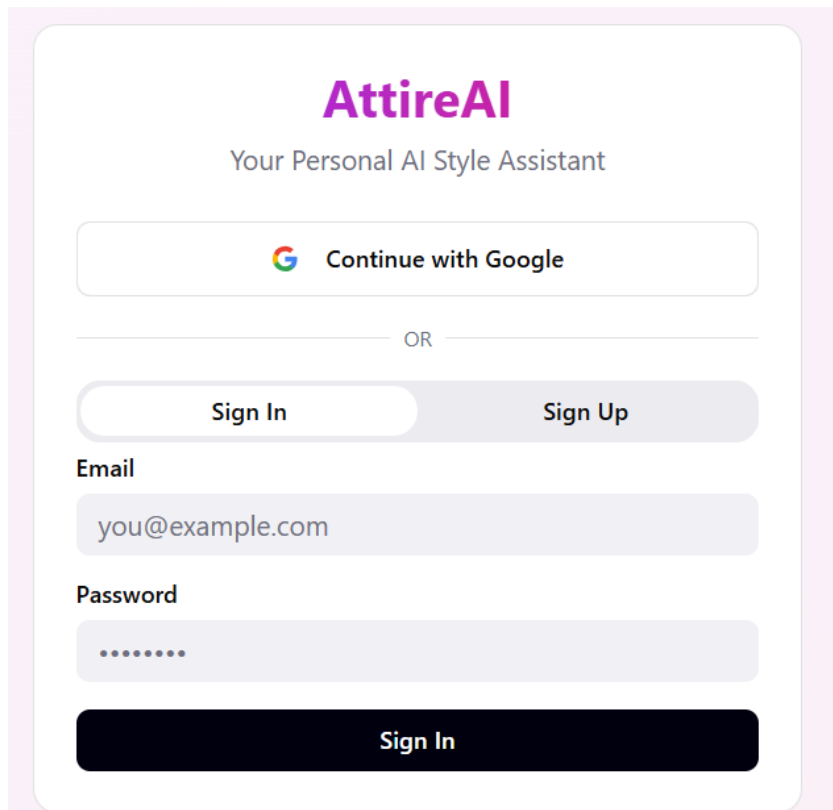


4.5 UI Mockups

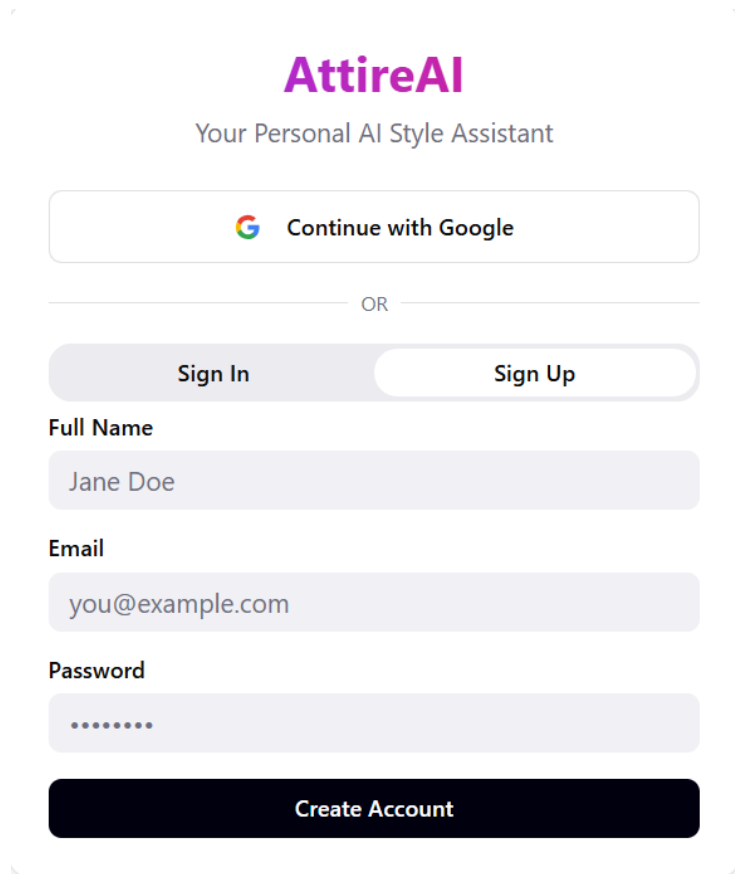
4.5.1 Sign Up/ Login Page

This screen will appear when users first access the AttireAI application. It serves as the entry point to our platform, allowing users to either create a new account or log into an existing one. Users can also choose Continue with Google.

Sign In Tab: Users can log in using their registered email address and password. Upon successful authentication, users will be redirected to their personalized dashboard, where they can access body measurements, style preferences, and outfit recommendations.

The image shows a mobile app login and sign-up screen for 'AttireAI'. At the top, the app's name 'AttireAI' is displayed in a bold, purple font, followed by the tagline 'Your Personal AI Style Assistant' in a smaller, grey font. Below this, there is a 'Continue with Google' button featuring the Google logo. A horizontal line with the word 'OR' in the center separates this from the email/password login section. This section has two tabs: 'Sign In' (which is active and highlighted) and 'Sign Up'. Under the 'Sign In' tab, there are two input fields: 'Email' with the placeholder text 'you@example.com' and 'Password' with a masked input (dots). At the bottom of the form is a large, dark blue 'Sign In' button.

Sign Up Tab: New users can create an account by providing their full name, email address, and a secure password.



The image shows a login and sign-up form for 'AttireAI'. At the top, the logo 'AttireAI' is in purple, followed by the tagline 'Your Personal AI Style Assistant'. Below this is a 'Continue with Google' button with the Google logo. A horizontal line with 'OR' in the center separates this from the next section. This section has two buttons: 'Sign In' (highlighted with a grey background) and 'Sign Up'. Below the buttons are three input fields: 'Full Name' with the text 'Jane Doe', 'Email' with the text 'you@example.com', and 'Password' with a masked input (dots). At the bottom is a large black button labeled 'Create Account'.

AttireAI
Your Personal AI Style Assistant

 Continue with Google

OR

Sign In Sign Up

Full Name
Jane Doe

Email
you@example.com

Password
.....

Create Account

For security reasons, if a user enters incorrect credentials during login, the system displays a generic error message such as "Invalid email or password".

4.5.2 Homepage / Body Measurements

This is the main dashboard that users see after successfully logging in. The page greets users with a personalized welcome message displaying their name, creating a friendly and engaging experience.

AttireAI

Home

Recommendations

Profile

Welcome back, Yuanfei Song! 🖐️

Let's find the perfect outfit for you

1. Measurements

2. Preferences

3. Get Outfits

📏 Body Measurements

Enter your measurements manually or use the camera for automatic detection

Manual Entry

Camera Capture

Chest (inches)

38

Waist (inches)

32

Hip (inches)

36

Inseam (inches)

30

Shoulder Width (inches)

17

Sleeve Length (inches)

33

Height (inches)

70

Weight (lbs)

165

Save Measurements

Navigation Bar: The header contains the AttireAI logo on the left and a navigation menu on the right with three main sections: Home, Recommendations, and Profile. This consistent navigation bar appears across all pages, allowing users to easily access different parts of the application.

AttireAI

Home

Recommendations

Profile

1. Measurements

2. Preferences

3. Get Outfits

Body Measurements Section: This is the first and most critical step in personalizing outfit recommendations. Users have two options for inputting their body measurements:

- **Manual Entry:** Users can directly input their measurements into clearly labeled fields, including Chest, Waist, Hip, Inseam, Shoulder Width, Sleeve Length, Height, and Weight. All measurements are displayed in standard units (inches for dimensions, pounds for weight) to ensure consistency. This option is ideal for users who already know their measurements or prefer precise control over their data.

 **Body Measurements**

Enter your measurements manually or use the camera for automatic detection

Manual Entry

Camera Capture

Chest (inches)

38

Waist (inches)

32

Hip (inches)

36

Inseam (inches)

30

Shoulder Width (inches)

17

Sleeve Length (inches)

33

Height (inches)

70

Weight (lbs)

165

 Save Measurements

- **Camera Capture:** For users who don't know their exact measurements or want a more convenient experience, the application offers AI-powered automatic body measurement detection. Users simply position themselves in front of their device's camera and click "Start Camera" to begin the detection process. On-screen guidance helps users achieve optimal positioning for accurate results.

 **Body Measurements**

Enter your measurements manually or use the camera for automatic detection

Manual Entry

Camera Capture



Position yourself in front of camera

 Start Camera

4.5.3 Homepage / Style Preferences

This is the second step in the outfit recommendation workflow, where users personalize their style profile to help our AI generate more accurate and relevant outfit suggestions.

Fashion Styles Selection: Users can select multiple fashion styles that match their personal taste. Each style is presented as a clickable card with a brief descriptive subtitle explaining the aesthetic. Users are encouraged to select all styles that apply to them, allowing the AI to provide more diverse outfit recommendations that suit different moods and occasions.

Skin Tone & Hair Color: This section captures the user's coloring information, which is essential for recommending clothing colors that complement their natural features. Both skin tone and hair color options are displayed with intuitive color swatches for easy visual identification. For users who are unsure about their coloring, the "Auto-Detect" button provides a convenient alternative by using the device's camera to automatically analyze and detect the user's skin tone and hair color.

Once all preferences are selected, users click "Save Preferences" to store their choices and proceed to the final step, where they can generate personalized outfit recommendations.

 **Style Preferences**

Help us personalize your outfit recommendations

Fashion Styles (Select all that apply)

Minimalist
Clean lines, neutral colors

Streetwear
Urban, trendy, bold

Classic
Timeless, elegant

Bohemian
Free-spirited, relaxed

Athletic
Sporty, comfortable

Elegant
Sophisticated, refined

Skin Tone & Hair Color

📷 Auto-Detect

Skin Tone

☐ Fair

☐ Light

☐ Medium

☐ Olive

☐ Tan

☐ Deep

Hair Color

☐ Blonde

☐ Brown

☐ Black

☐ Red

☐ Gray

☐ Other

 **Save Preferences**

5. Summary

This design document outlines the architecture and implementation plan for AttireAI, an AI-powered personal styling web application. Key architectural decisions include:

- **Frontend**: Next.js 14 with TypeScript on Vercel
- **Backend**: FastAPI (Python) for AI/ML processing

- **Database**: MySQL for structured user data
- **Authentication**: Firebase Auth for secure, social-enabled login
- **Storage**: AWS S3 for user images
- **AI Models**: MediaPipe for body detection, diffusion models for virtual try-on

The modular architecture allows for independent scaling of compute-intensive AI features while maintaining a responsive user experience. The VIP tier with virtual try-on provides a premium feature that differentiates AttireAI from competitors.

Document Version: 1.0

Last Updated: January 2025

Team 11 - AttireAI